

STATUS

Migration status

Where things stand

Done

The three verification gates

What the duplicate gate found

What hardware found that QEMU could not

What made the test deduplication possible

Three defects found and fixed on the way

Decisions — all resolved

Next step

Sanitizers on the POSIX port

Things a fresh session should not rediscover

Migration status

Updated: 2026-09-22 · **Phase:** step 5 — **complete**. Five architecture projects exist; twelve board configurations build.

This file is the cold-start entry point. Read it, then docs/specs/2026-09-20-micro-os-plus-iii-smp-unification-design.md for the full design and the measurements behind it.

Where things stand

Steps 1 and 2 are complete. Six repositories exist; both A-profile architecture projects build all 24 of their targets from a single copy of every test, and all twelve tests pass on a Raspberry Pi Zero 2 W on **both** ports — each measured on that port, neither inferred from the other.

Step 3 is in progress. The AArch32 port now carries a second board, the Luckfox Lyra B (RK3506, 3× Cortex-A7): 19 targets build, and both ports still pass their QEMU suites (11/1/0 each) after the restructure it required. What that second board changed, and the seven portability defects it exposed in shared code, are written up in [aarch32-second-board.md](#).

The Lyra is a hardware-only board, decided and closed. QEMU models none of the RK3506's own blocks, so an emulated Lyra would be a generic Cortex-A7 wearing the name — and the DWC2 gadget, the SD host and the Cortex-M0 mailbox, which are what make this board worth having, are exactly the parts not modelled. Its board.cmake sets no UOS_BOARD_LINKER_QEMU, so it builds one image per test rather than two, and test/qemu.sh says so if asked. This is not a gap to close later; do not reopen it.

Step 5 is complete. micro-os-plus-iii-posix-arch exists: SMP on the development machine, one host thread per CPU, preemptive. One board, native, eleven tests, **11 passed / 0 skipped / 0 failed** — nine SMP object tests carried unchanged at OS_NCPU=4, and mutex-stress and rtos-apis from the kernel's own tests/sources/ at OS_NCPU=1. It is the only one of the four ports that is new work rather than a migration, and it has now earned its keep twice: it found a real SMP race in the kernel's mutex (see below), and, the first time it was run under ASan, a dangling thread name that has shipped in every copy of smp-pro-cons-test since the test was written (see *Sanitizers* below). Written up in [posix-arch-port.md](#).

Step 4 is under way. micro-os-plus-iii-cortexm exists, with six boards: three STM32F4 at OS_NCPU=1 and three RP2350 boards at OS_NCPU=2 — the Raspberry Pi Pico 2, the WeAct RP2350B with 8 MB of PSRAM, and the Pi-Zero RP2350B. The STM32 boards are the first OS_NCPU=1 boards in the workspace, so they are also the first to build the kernel's **non**-SMP branch — every board before them had more than one CPU. Nothing on this port has run on hardware yet. The port, its six boards and where its SMP lives are written up in [cortexm-port.md](#).

```
TMP7/
├─ micro-os-plus-iii-smp/          kernel + shared build rules +
test runners
|   ├─ src/ include/              the kernel, upstream path-for-
path
|   └─ port/smp-common/          os-decls.h, shared by both ARM
ports
|   └─ test_smpl/                the two runners -- see test-
smpl.md
|       └─ run-qemu.sh            the shared QEMU suite runner
|       └─ run-host.sh           the same, for the POSIX host
|       └─ run-hw.sh             the shared hardware session
runner
|   └─ cmake/                    toolchains + uos_add_app
|   └─ tools/                    the verification gates
|       └─ verify-kernel-compiles.sh    kernel builds on a
port's headers
|       └─ verify-no-duplicate-sources.py spec Section 9,
cross-repo
|       └─ verify-no-absolute-paths.sh   spec Section 9
└─ micro-os-plus-iii-devices/    flatfs, FatFs; SD per SoC
```

```

(BCM2837, RK3506,
|
| native – an image file);
| silicon support per SoC
(BCM2837, STM32F4,
|
| micro-os-plus-iii-aarch64/ RP2350)
|   | src/ include/ ARMv8-A port
|   |   | test/ ARMv8-A, every board
|   |   | everything board- or test-
specific
|   |   | CMakeLists.txt one loop over test/<board>/*/
|   |   | hw.sh qemu.sh dispatchers; BOARD picks the
directory
|   |   | boards/{rpi-zero-2w,rpi3b}/ board.cmake, include/,
src/,
|   |   | linker, OpenOCD, hw.sh,
qemu.sh
|   |   | rpi-zero-2w/ rpi3b/ each board's test applications
|   |   |   | build*/ the build trees
|   | micro-os-plus-iii-aarch32/ ARMv7-A port
|   |   | src/ include/ ARMv7-A, every board – incl. the
scheduler
|   |   | test/ same shape
|   |   | boards/{rpi-zero-2w,rpi3b,luckfox-lyra}/ BCM2837 ·
BCM2837 · RK3506
|   |   | rpi-zero-2w/ rpi3b/ luckfox-lyra/ 12 · 12 · 19
applications
|   | micro-os-plus-iii-posix-arch/ POSIX host port – host threads
as CPUs
|   |   | src/ include/ the CPU model, the scheduler
half, the
|   |   | fault reporter, the free store
|   |   | test/
|   |   | run.sh dispatcher; BOARD picks the
directory
|   |   | boards/native/ board.cmake, include/, src/,
run.sh
|   |   | native/ 10 applications (9 at NCPU=4, 1
at NCPU=1)
|   | micro-os-plus-iii-cortexm/ Cortex-M port – M4F and M33, 1
and 2 CPUs
|   |   | src/ include/ upstream's single-core core
(STM32 boards)
|   |   | src/rtos/os-core-rp2350.cpp
|   |   | include-rp2350/ the same core plus an SMP branch
(pico2)
|   |   | test/ same shape
|   |   | boards/{nucleof411,weactf411,weactf412}/
|   |   |   | STM32F411RE · F411CE · F412RE
|   |   | boards/{pico2,pico2-rp2350b-psram,pico2-pizero}/
|   |   |   | RP2350 4 MB · RP2350B +PSRAM ·
RP2350B
|   |   | the last two include pico2's

```

```

board.cmake
|      └─ <board>/          1 · 2 · 2 · 12 · 14 · 10
applications
└─ micro-os-plus-iii-smp-old/  READ ONLY – the migration source

```

Outside `src/` and `include/` – which are the ISA and nothing else – an architecture project is all `test/`: the boards, their tests and their build trees. How that is laid out and how to run it is `tests-in-aarch32-aarch64.md`.

Migration source (read-only)	TMP7/micro-os-plus-iii-smp-old
Remotes	GIT/micro-os-plus-iii-{smp,devices,aarch32,aarch64,cortexm,posix-arch}.git
Old remote	GIT/micro-os-plus-iii-smp-old.git

An architecture project holds **no copy** of the kernel or the devices repo. CMake resolves both as sibling directories, overridable with `-DUOS_SMP_DIR=` and `-DUOS_DEVICES_DIR=`. Whoever builds one clones what it needs. One working copy of the kernel serves every architecture project on the machine, so an edit to it is visible to all of them at once. The dependency runs one way: neither the kernel nor the devices repo knows an architecture project exists.

Done

1. **Remotes reorganised**, old tree analysed and measured, design spec written. Every figure in the spec came from the tree, not estimation.
2. **Step 1 — skeleton and kernel**. Kernel vendored once, flattened to the repository root so it matches upstream path-for-path. Three toolchains share one preamble. `uos_add_app` replaces 214 Makefiles.
3. **Devices split out** (`micro-os-plus-iii-devices`). All seven sources compile for armv7-a *and* armv8-a from one copy; ~12,000 duplicated lines collapse to one copy.
4. **Step 2a — the twelve test applications unified**. 15,507 lines of application C++ become 7,610. Nothing ISA-specific is left in a test.

5. **Step 2b — both ARM architecture projects.** Each builds every board's tests from one loop over `test/${BOARD}/*`, in two variants. No test name and no board name appears in either port's build files. 24 targets per Pi board. The Lyra is 19: it is hardware-only, so it builds one image per test rather than two.
6. **QEMU suites green on all four board/port pairs.** Both `ports × {rpi-zero-2w, rpi3b}` give 11 passed, 1 skipped, 0 failed, with gcc 15 and **one suite at a time**. `usb_test` skips by design — QEMU emulates no USB device mode — as the predecessor suite also recorded.

aarch32 + rpi3b was briefly recorded here as 10 / 1 / 1, with `smp-mat-sdcard-test` TIMEOUTing after 2000 s. That was wrong, and the cause was procedural: the four suites had been launched two at a time, which is exactly what the “QEMU suites must run one at a time” rule below forbids, and the stall had that rule's documented signature — a worker frozen mid-loop with no fault. Run alone, that test passes in 113 lines, and the whole suite passes 11/1/0. **Re-run before investigating** is in that rule for a reason; it was not followed.
7. **Documentation.** `docs/smp-construction.md` (the port contract, the division of labour, and the defines per folder) and `docs/building-aarch32-aarch64.md`, both with PDFs, rendered by a single `docs/md2pdf.py` that replaces three near-identical copies.
8. **Hardware green on both ports.** All twelve pass on a Zero 2 W over a J-Link, `usb_test` included — the one QEMU can never run. `test_smpl/run-hw.sh` plus a ~40-line `test/hw.sh` per port replace the predecessor's 48 per-test runner scripts.
9. **Step 5 — the POSIX port.** A new SMP implementation, OS_NCPU host threads as CPUs, preemptive from the start: per-CPU `timer_create + SIGEV_THREAD_ID`, `pthread_sigmask` as per-CPU interrupt masking, `pthread_kill` as the IPI, the context switch performed inside the signal handler. Migration is allowed and native TLS is banned. 11/0/0.
10. **The two Section 9 gate scripts.** `verify-no-duplicate-sources.py` pools the tracked sources of every repository and compares them — running it one repository at a time would miss precisely the duplication D9's multi-repo layout permits. `verify-no-absolute-paths.sh` looks for machine-specific roots. Both were checked against injected violations, not only against a clean tree.

The three verification gates

gate	command	state
kernel compiles on a port's headers	tools/verify-kernel-compiles.sh <port>/include	61/61 on every port
no duplicate sources, across repos	tools/verify-no-duplicate-sources.py	22 unexplained pairs — see below (493 sources compared)
no machine-specific absolute paths	tools/verify-no-absolute-paths.sh	PASS (105 files)

The duplicate gate has **three** outcomes, not two, because this workspace duplicates some code on purpose and a flat pass/fail would have to lie about it:

- **exempt** — never compared: upstream's tests/, vendored xpacks/, TinyUSB, and ARM's own CMSIS core headers (415 files). One consequence is worth knowing: the two single-core tests the POSIX board carries, mutex-stress and rtos-apis, are near-copies of tests/sources/, and the gate cannot see the pairing because the originals are on the exempt side. It is deliberate — tests/ is carried as shipped (spec Section 10) — but it is not the gate proving those copies are justified.
- **expected** — reported and counted, but not a failure: *the same test file carried by two boards*. Every board owns its tests, deliberately, so that changing a test reaches exactly one board. **257 pairs**. The spec's Section 9 says "target: zero findings" and was written before that decision; this is where the two are reconciled, in the open rather than by a silent exemption.
- **unexplained** — everything else, and the only thing that fails the gate. **22 pairs**, listed in the next section.

The absolute-path gate skips exactly two things: upstream's tests/, and **itself**. The second is not tidiness — its header comment spells every denied root out loud, so the moment it was committed it matched itself four times and could never pass again. A real absolute path added to that one file would go uncaught; that is the price of the denylist being readable, and it is the only exemption. Both gates have been negative-tested against an injected violation, before and after that change.

What the duplicate gate found

Nothing in the kernel and nothing in a port's scheduler. The 22 pairs are, by kind:

what	pairs	verdict
aarch32/aarch64 hw_result.hpp and board-contract.cpp, byte-identical	2	real — candidates for the kernel repo
FatFs: devices/ fatfs/ff.c vs the Lyra board's fatfs- cpp/ff.cpp (0.993, 5727 lines)	1	real , and the largest
RK3506 SD driver: devices/soc/rk3506/ sdmmc.* vs smp_test_int4's private copy	2	real — a test carrying its own driver
Lyra USB glue: the board's usb/src/* vs smp_test6/ smp_test7's copies	6	real — same shape as above
Lyra interrupt tests smp_test_int3/int4 sharing host and USB sources	2	real
cortexm syscalls.c, four boards	3	real , small
RP2350 smp-test- nested- clock{, _200, _250}, differing only in a clock constant	3	a board_test_defines() job
pico2-pizero sc- test-ko vs smp-test- ko (0.912)	1	the single-core/SMP pair; arguably board_test_ncpu()
cortexm os-decls.h vs include-rp2350/... (0.936)	1	known and deliberate — the two port cores, see cortexm-port.md
TinyUSB tusb_config.h cdc vs hid (0.852, 26 lines)	1	class-specific by design

Every one of them is in the Lyra's or cortexm's test trees — that is, in the parts of steps 3 and 4 that have never run on hardware. None is in code any passing suite exercises.

What hardware found that QEMU could not

Five real defects, all in paths the emulator does not reach:

Defect	Fix
Every blinking test drove GPIO16 , so nothing lit	LED_PIN is a board fact now, 29 on the Zero 2 W, set once for every test. Only usb_test had ever set it — and the predecessor Makefiles had the same hole.
usb_test answered a PUT after streaming its hexdump and listing	send_reply moved to immediately after the store. A completed store looked like a hang.
Console posts blocked the USB service thread, so bulk OUT was not re-armed	try_send, with console_dropped in the tally. Diagnostics must never throttle the protocol.
LIST compared file names case-sensitively	FatFs on a boot card has no long names and returns XFER.BIN; flatfs under QEMU keeps the name as sent.
kOutChunkMax overflowed PKTCNT — a 1 MiB transfer stored 61440 bytes, silently truncated	D{I,0}EPTSIZ is bounded by two fields. GHWCFG3 = 0xff000e8: XFRSIZ 19 bits but PKTCNT 10 bits = 1023 packets , so 65472 bytes at full speed. The chunk derives from the packet limit now, with static_asserts on both fields.

What made the test deduplication possible

The ISA seam in the applications turned out to be tiny, and mostly not a seam:

what the two copies disagreed on	resolution
dsb vs dsb sy, dmb vs dmb ish	not a difference — the explicit forms assemble on ARMv7-A too. Verified with both assemblers.
cpsie i vs msr daifclr, #2	already a kernel API: interrupts::uncritical_section::enter(), implemented by every port

what the two copies disagreed on	resolution
mrrc p15...c14 vs mrs cntpct_el0	already in the port's timer_arm.hpp; the tests were re-reading the register themselves
"(AArch64)" in banners	PORT_BANNER_ISA / PORT_BANNER_CPU, beside the existing PORT_BANNER_LONG/SHORT

Two pieces of per-application boilerplate were also collected: test-smp-boot (idle stacks, idle body, smp_install_boot_threads(), in ten of twelve tests) and test-console (the mutex-guarded print helpers, in three). The shared boot helper generates thread names from OS_NCPU instead of listing four, so it no longer assumes a four-core BCM2837.

Three defects found and fixed on the way

- **The kernel exported all 63 sources as one target**, but the proven rpi builds compiled exactly 37. Linking the rest breaks the build: posix-io/c-syscalls-posix.cpp declares read/write returning ssize_t, newlib declares them returning int. micro-os-plus::iii is now that measured 37-source core; posix-io, drivers, generic startup, newlib-reent, semihosting and the three trace backends are opt-in targets. Two entries were headers listed as sources and are gone — which is why 63 entries were always 61 compiled files.
- **The Cortex-A7 port's os-decls.h was missing volatile on lock_state[],** which several cores read and write. The shared copy in port/smp-common/ is the correct one, so adopting it fixes that port rather than merely deduplicating it.
- **mutex::internal_try_lock() dereferenced a null owner.** In the priority-inheritance path it opens a scheduler::uncritical_section — which *releases* the kernel lock, because priority_inherited() ends in a yield — and then wrote through owner_. While that section is open the owner can finish its own unlock() on another CPU, and internal_unlock() ends by setting owner_ to nullptr. Every SMP port has the window; it is rare on hardware because scheduler::unlock()/lock() is a handful of instructions, but on the POSIX host it is two pthread_sigmask() system calls, so smp_test2 faulted in 21 runs out of 25 at four CPUs. The fix captures the owner under the lock and re-checks ownership inside the uncritical section. **This is what a synthetic SMP host is for**, and it is the whole justification for step 5.

Decisions — all resolved

Spec Section 11 has the table; nothing is open. Revision 2 set the shape: architecture-specific ports (not target-specific), six repositories, architecture projects independent and *outside* the main repo, devices in their own repo, cortexm and posix-arch both to become SMP.

Next step

Step 2 is done. *Gate:* all 24 targets build (**met, both ports**); QEMU suites pass (**met, 11/1/0 both ports**); hardware tests pass on the Pi (**met, 12/12 both ports**); no device or test source exists in more than one repo (**met**).

Step 3 (aarch32 gains the RK3506) is **under way**. *Gate:* exception_handler.cpp shared unmodified by both boards (**met**); the board builds (**met, 19 targets**); neither existing port regressed (**met, QEMU 11/1/0 on each**). **The board has been run on hardware** — the maintainer confirms it, and this workspace retains one log of it, test/build-lyra/test/.hw-logs/smp_test5.log, which shows all three A7 cores up, the heartbeat running past 42 s and a clean shutdown. What is *not* recorded here is a per-test verdict table: the logs for the other eighteen targets are not in this tree, so this document does not claim pass/fail for them one by one. The earlier statement that the six RK3506-specific tests “have never been run” was wrong and is withdrawn.

Emulating the Lyra is **not** open. It was considered and closed: see above.

Step 4 (cortexm) is **under way**. *Gate:* no app source exists more than once; STM32 boards pass at OS_NCPU=1; pico2 passes at OS_NCPU=2. The repository exists and all six boards build — three STM32F4 (5 applications) and three RP2350 (36 images). **Nothing on this port has been run on hardware**, so the two pass clauses of the gate are not met and are not claimed. What the ELF's do show: the single-core tests carry no launch_core1 and no per-core idle table while the SMP ones carry both, the PSRAM tests put .bss and the heap at 0x11000000 while their board-mates keep them in internal SRAM, each USB image carries only its own class driver, and the two kernel-less probes carry zero os::rtos symbols where their board-mates carry 145.

The spec framed this step as “merge pico2’s SMP core with the STM32 boards”. Measurement changed the shape, for the better:

- pico2’s core is **not portable Cortex-M SMP**. Its kernel lock is the RP2350’s SIO hardware spinlock 0, its IPI the SIO inter-core FIFO on IRQ 25, its CPU index SIO_CPUID. All three are silicon, so in this layout they are **board** code — and cortexm gained SMP by a board arriving, exactly as the Lyra’s GIC-400 SGI and the Pi’s are board code.
- The four “pico2 variants” differ in **one** thing, and it is that same kernel lock: pico2 uses the SIO spinlock, pico2-std Peterson’s algorithm in software (the RP2350 has no global exclusive monitor, so LDREX/STREX does not work across the cores), pico2-sdk the Pico SDK — and pico2-sdk-min has **byte-identical** port files to pico2-sdk, so it is not a separate port at all. **Only pico2 is of interest**; the other variants are out of scope and are not open work.
- **What is uncarried inside pico2 is not a duplicate — it is a capability.** Of that tree’s 33 application directories, cortexm carries 16: the kernel object tests, smp-test0...smp-test5, the nested-interrupt set and the two USB images. The other 18 are one subject area, and cortexm has none of its infrastructure (find . -iname '*xip*' -o -iname '*loader*' returns nothing but the PSRAM board files):

group	dirs	source lines
XIP: smp-test-xip, -corelpark, -fs, plus shared-xip and xip-glue	5	~7,400
XIP loader: smp-test-xip- loader-{apps,apps-mos+ +,dyn,dyn-no-list,static}	5	~11,500
RAM loader: smp-test- loader- {dynamic,over,reloc,static}	4	~1,500
pizero PSRAM execution: pizero-psram-exec{,-main,- main_1,-main_2}	4	~1,550
mem-diag, smp-test- nested_latency	2	~550

About 22,000 lines in total. Execute-in-place and a dynamic loader are a different piece of work from carrying another test, and neither was in step 4’s gate. Recorded so the gap is not mistaken for a finished migration.

Step 5 (posix-arch) is **done**. *Gate*: the existing upstream tests pass at OS_NCPU=1 (**met** — mutex-stress *and* rtos-apis, **both carried from the kernel's own tests/sources/**; the nine SMP object tests cannot be built single-core, because they use `thread::cpu_affinity()`); the SMP object tests pass at OS_NCPU>1 (**met, 9/9 at OS_NCPU=4**); no use of sigprocmask or setitimer remains (**met** — pthread_sigmask and per-CPU timer_create). Both ARM QEMU suites were re-run after the kernel mutex fix and after the shared test sources lost their last ARM assembly: 11/1/0 on each, unchanged. All four board/port pairs were run again after the dangling-name fix, one suite at a time: 11/1/0 on every one.

That the port really is SMP is **measured, not claimed**: four CPU-bound threads with no affinity set report 40 runs each on cores 0, 1, 2 and 3, and smp_test4's eight workers each accumulate thousands of runs on all four CPUs. The counter-example matters as much — the same four threads with a `sleep_for()` in the loop report core 0 every time, because only CPU 0 advances the kernel clock. See the cold-start facts below before reading anything into a per-core distribution.

Sanitizers on the POSIX port

Measured, not asserted — the whole eleven-test suite was built and run under each. `-DUOS_SANITIZE=<set>` on the native board; details in [posix-arch-port.md §21](#).

sanitizer	suite	reports	usable as a gate?
address	11/11 pass	0	yes
undefined	11/11 pass	29, all vptr, all in the kernel	yes , with <code>-DUOS_SANITIZE_NO_VPTR=ON</code> → 0 reports
thread	runs, still reaches RESULT: PASS	5,980 (3,775 races, 2,203 errno)	no

Three things a fresh session should not have to rediscover:

- **The port is annotated for the ASan fiber API** and has to be. Without `__sanitizer_start_switch_fiber / __sanitizer_finish_switch_fiber` around `swapcontext`, every context switch is a false stack-buffer-overflow. The save slot is a **local**, so it rides the outgoing thread's own stack — a per-CPU slot would be read by the wrong host thread after a migration, the same hazard as the deferred publish.
- **ASan's first run found a real bug, and it is upstream.** `smp-pro-cons-test` named its eight workers from a loop-local `char name[16]`, and `os::rtos::named_object` stores `const char* const`

name_ — the pointer, never a copy.
 scheduler::is_thread_allowed_on_cpu() then strcmp()s that dead stack slot on **every scheduling decision**. It is in micro-os-plus-iii-smp-old (rpi/.../64b/smp-pro-cons-test/main.cpp:524, :540) and therefore in all **six** shipped copies of the test — aarch32 × {rpi3b, rpi-zero-2w, luckfox-lyra}, aarch64 × {rpi3b, rpi-zero-2w}, and posix-arch/native. **All six are now fixed**, and the five ARM copies are still byte-identical to each other. Four emulated boards rebuilt and re-run, smp-pro-cons-test green on all four; the Lyra copy builds and its image carries the fix, but it is hardware and was not run. A sweep found no second instance — pool_thread_names, solver_thread_names and test-smp-boot.cpp's idle_name[] were already static.

- **The 29 UBSan reports are the kernel's intrusive-list sentinel idiom**, not a defect: clock_timestamps_list::link() downcasts the bare head_links to timeout_thread_node* to use as a loop terminator and only ever asks it for ->prev(). Upstream already wraps those casts in #pragma GCC diagnostic. If that count ever changes, something else changed. Also:
 -DCMAKE_CXX_FLAGS=-fno-sanitize=vptr does **not** suppress them — CMAKE_CXX_FLAGS is emitted before the target's options and the last -f[no-]sanitize= wins. That is why there is a UOS_SANITIZE_NO_VPTR option.

TSan needs __tsan_create_fiber / __tsan_switch_to_fiber (the same shape of work already done for ASan) and an errno save/restore around the switch in the tick handler. Until then it is a by-hand tool, not a gate.

Things a fresh session should not rediscover

- **Two RP2350 tests compile no kernel**, and that is the point of them: smp-test0 (dual-core bring-up) and exc-test (first-exception probe) run before a scheduler exists. uos_add_app() takes NO_KERNEL, the port exports a board-only micro-os-plus::cortexm-bare beside the full target, and BOARD_TEST_NO_KERNEL names them. Do not rewrite them as RTOS tests.
- **On the RP2350 boards, SMP is a per-TEST fact, not a board fact.** smp-test1 (single-core RTOS bring-up) and sc-test-ko (single-core kernel-object test) define OS_USE_SMP_SCHEDULER in no Makefile. uos_add_app() derives that define from NCPU, and board_test_ncpu() names the two tests. sc-test-ko is the only test on that silicon exercising the kernel's **non-SMP** branch.
- **The nested-interrupt tests need -g0 -mlong-calls on main.cpp alone.** They place handlers in .data to run from RAM instead of XIP flash. Without those two options the assembler stops

with Error: leb128 operand is an undefined symbol. A call from flash to a .data function is also out of a Thumb BL's reach.

- **pico2-pizero builds a different port branch.** It forces `__ARM_ARCH_8M_MAIN__` where the other two RP2350 boards force `__ARM_ARCH_7EM__`. The M33 is ARMv8-M Mainline and the predecessor's Makefiles for that board said so; the two branches have never been compared on the desk.
- **TinyUSB is carried once.** The predecessor kept `src/usb-cdc/` and `src/usb-hid/` byte-identical apart from one class driver each. The per-test include directory must come first — TinyUSB includes `tusb_config.h` by plain name.
- **smp-test-mini-a-usb-cdc-acm_agy is a byte-identical duplicate** of its sibling, `main.cpp` and Makefile both. Like `pico2-sdk-min`, it is not a separate thing.
- The four patched kernel copies are **byte-identical** (`diff -rq`).
- The kernel differs from pristine upstream in exactly **6 files**, all under `rtos/`. That is the entire SMP delta.
- **Upstream fork point**, preserved because the spec drops BASE: `micro-os-plus-iii @ c47f806b57f8b9b870ec7b89ded663dec354df96` (branch `xpack-development`), `micro-os-plus-iii-cortexm @ 687e975caa298519cb214d4b5a9774c48f784816`.
- **src/rtos/os-core.cpp exists twice on purpose.** The kernel's is the portable SMP scheduler; each port has its own at the same relative path with the bring-up, the kernel lock and the context-switch hooks. Both are compiled, exactly as the old Makefiles did. They do not collide.
- **A port needs its machine flags at link time too**, not just compile time. Without them the driver picks the wrong multilib and every AArch32 link fails with "uses VFP register arguments".
- **Rebuild the port you are about to test.** A shared test source is still two separate binaries. `usb_test` on AArch32 reproduced a defect that had already been fixed, purely because only the AArch64 target had been rebuilt; the symptom was indistinguishable from the original bug.
- **A hardware run is one test per power cycle.** It is `load_image` into RAM over the previous test's leftovers, and the Pi has no SRST — the Cortex-A53 debug target has no reset method a script can drive. `run-hw.sh` refuses a suite for this reason.
- **A hardware budget is not the test's own duration.** It is dominated by semihosting traps, which scale with how much a test prints. `smp_test4` reaches its verdict at `t=9597` ms of target time and still needs more than 120 s of wall clock.
- **Invalid ACK (0) in DAP response is not a firmware fault.** The debug link gave up; once it is gone nothing services the semihosting traps, so every core halts inside one and it reads like a hang. Drop the JTAG clock with `U0S_HW_ADAPTER_KHZ=1000` and check the supply. The runner reports this as `DEBUG LINK LOST`.
- **QEMU suites must run one at a time.** Running two four-core suites concurrently, or alongside a build, starves a vCPU: `smp_test2` stalled with three workers finished and the fourth

frozen mid-loop. On an idle host the same binary passes 5 out of 5. A stalled worker with no fault is contention, not a bug — re-run before investigating.

- **The AArch32 suite runs on raspi3b with the boot shim, never raspi2b.** The port is built `-mcpu=cortex-a53`, so its load-acquire instructions are undefined on the raspi2b Cortex-A7 and it faults at the first one. QEMU's raspi3b starts its cores in AArch64, so a 20-line stub drops to AArch32 and jumps to the image. Hardware needs none of this: `config.txt` sets `arm_64bit=0`.
- **rtk's diff reports "Files are identical" for files with different MD5s.** Seen on `sd.cpp`, `bcm2837.hpp`, `bcm_irq.hpp`. Use `md5sum` or Python `difflib` for anything that matters.
- rtk silently dropped the `-u` flag from `git push -u`; tracking was set with `git branch --set-upstream-to`. Expect this on new branches.
- Do **not** add a blanket `*.html` ignore rule — the kernel ships three doxygen templates as `.html`.
- There are **214 tracked Makefiles**, not the 24 an early count suggested.
- A test app is only ~5 tracked files; the ~100 others per directory are untracked build artifacts.
- **The SMP port contract is small.** The whole kernel patch is 379 lines and asks a port for only: `OS_USE_SMP_SCHEDULER`, `OS_NCPU`, `port_cpu_id`, `port::scheduler::switch_stacks`, `port::stack::element_t`, plus a kernel lock and an IPI.
- **cortexm SMP is not from scratch, and it is not portable.** `pico2's os-core.cpp` is a complete dual-core Cortex-M33 SMP port, but its lock, IPI and CPU index are RP2350 SIO registers. It is 91% the same file as upstream's single-core core, and its port headers already handle every M profile (6M, 7M, 7EM, 8M_MAIN) with the SMP parts gated on `OS_USE_SMP_SCHEDULER`. They are **not merged**: the three STM32 boards are hardware-proven on upstream's core, so a board names the core it was proven with (`UOS_BOARD_PORT_CORE`).
- **pico2 forces -D__ARM_ARCH_7EM__** although the M33 is ARMv8-M, exactly as the predecessor's Makefiles did. The v7E-M path is the one that board was brought up on; letting the compiler pick `__ARM_ARCH_8M_MAIN__` would change the scheduler under a working board.
- **pico2's smp-test0 compiles no kernel.** Nine files, none of them `µOS++` — it is the bare-metal dual-core bring-up test that runs before any scheduler exists. `uos_add_app()` always links `micro-os-plus::iii`, so it is parked in `test/pico2/.pending/` rather than rewritten.
- **Flashing the Pico 2: resume cm1 BEFORE cm0.** With `USE_SMP 0` the `rp2350` target exposes two targets and `reset init` halts both, while one `resume` resumes only the current one. A debug-halted core 1 will not boot from the PSM reset `launch_core1()` issues, so the launch hangs forever on the bootrom readiness word. Also `reset init`, never a bare reset —

that does not re-run the bootrom/XIP setup and the new image never boots.

- **The STM32 boards are the workspace's only OS_NCPU=1 boards.** They are therefore the only ones that build the kernel's non-SMP branch at all.
- **posix-arch was pristine upstream v1.0.1** — single host thread, ucontext coroutines, cooperative only — and is now a preemptive SMP port. The three POSIX defects D12 names are all fixed: pthread_sigmask for sigprocmask, per-CPU timer_create + SIGEV_THREAD_ID for setitimer(ITIMER_REAL), and **native TLS is banned** rather than threads being pinned — no port or application state may live in errno or a thread_local across a switch point.
- **The SMP picker reads th->context_.port_.stack_ptr directly,** and the design spec does not say so. Null means “not safe to claim”. The POSIX port puts stack_ptr first in its thread context and uses it as the same flag, published not by the CPU that leaves a thread but by **whoever arrives next on that CPU** — because by the time swapcontext() returns, this CPU is already running the incoming thread. Same hazard and same answer as AArch64's _smp_pub_addr/_smp_pub_val; no kernel change was needed.
- **On the POSIX host the tick handler runs with no SA_ONSTACK,** on the µOS++ thread's own stack, so the signal frame migrates with the thread. That is why OS_INTEGER_RTOS_MIN_STACK_SIZE_BYTES is 32 KiB there. The *fault* handler is the opposite case and does use SA_ONSTACK.
- **The POSIX free store is first_fit_top, not glibc malloc.** A µOS++ thread can be preempted inside an allocation and resumed on a different host thread, and glibc's arena lock would then be released by a thread that never took it.
- **hw_result::ok()/fail() flush stdio before _exit() on the host.** _exit() does not, and to a pipe stdout is fully buffered — a carried upstream test ran, passed, and printed nothing at all.
- **The POSIX fault reporter prints the faulting pc, the image's load bias and a backtrace,** so addr2line -e <image> <pc-bias> names the line. It is how the mutex race above was found, after three speculative fixes had resolved nothing.
- **Wake-ups on the POSIX host are biased towards CPU 0, and that is not a bug.** Only CPU 0 calls os_systick_handler() — the BCM2837 arrangement, where all four cores take the 1 ms PPI but one advances the kernel clock — so a thread released by sleep_for(), a timer or a timeout is normally re-picked by CPU 0 before any other CPU's tick comes round. **CPU-bound work spreads evenly; clock-bound and I/O-bound work does not.** Measured: four CPU-bound threads report 40 runs each on c0/c1/c2/c3 with no affinity set anywhere, while the same four with a sleep_for() in the loop report core 0 every time. Both are correct. This is also why smp-pipeline-test prints Produced by Core: c0=3275 c1=0 c2=0 c3=0, and why smp_test4 — whose workers are CPU-bound — shows every worker on every CPU. **Before suspecting the**

scheduler because a test reports all its work on core 0, check whether its threads are CPU-bound or clock-bound. Distributing the clock across CPUs would change this and would also stop the port resembling the silicon it stands in for; do not “fix” it.

- **On the POSIX host, install the per-core idle threads BEFORE releasing the cores:** `smp_install_boot_threads()` then `smp::start_secondary_cores()`, never the reverse. A released CPU enters `reschedule()` at once, and with no idle thread of its own it finds nothing to run and aborts with `!!! no ready thread and no idle thread on CPU 1 !!!`. Every carried test already does it in that order.